Virtualization

Understand hypervisors, CPU and memory overcommit, virtual disks and snapshots, virtual networks and live migration, and troubleshoot a KVM host.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What virtualization is and the types of hypervisor (14 min)
2. CPU, memory and overcommit (15 min)
3. Virtual disks, snapshots and templates (14 min)
4. Virtual networking: bridges, VLANs and NAT (14 min)
5. Live migration, high availability and troubleshooting (15 min)
6. Virtualization operations: capacity, overcommit and noisy neighbours (12 min)
7. Virtualization operations: libvirt and KVM day-two work (13 min)
8. Virtualization operations: guest performance troubleshooting (12 min)
9. VM backup, replication and recovery (12 min)
10. VM templates, cloud-init and image pipelines (13 min)
11. GPU passthrough and device assignment (12 min)
12. Hypervisor security and isolation (12 min)
Cheat sheet

Lesson 1. What virtualization is and the types of hypervisor

14 minutes. Understand how one physical server runs many virtual machines, and how KVM, VMware, Hyper-V and Proxmox differ.

A server with 64 CPU cores and 512 GB of memory usually runs far below its limits when it hosts one application. Virtualization lets one physical machine, the host, run many virtual machines, called guests, each with its own operating system, and shares the hardware between them.

What you will learn

- What a hypervisor does and why modern CPUs help it
- Type 1 and type 2 hypervisors, with KVM, ESXi, Hyper-V and Proxmox as examples
- How a virtual machine differs from a container
- The pieces of a KVM host: kernel module, QEMU, libvirt

What a hypervisor does

A **hypervisor** is the software that creates virtual machines and hands each one a share of CPU, memory, disk and network. A guest believes it owns a whole computer. The hypervisor keeps guests apart, so a crash or an attack inside one guest does not reach the others.

Fast virtualization depends on **hardware support**. On Intel this is VT-x, on AMD it is AMD-V. These CPU features let guest code run directly on the processor, and only a small set of sensitive operations is trapped by the hypervisor. Without them, virtualization is slow or needs tricks.

```
$ grep -E -o 'vmx|svm' /proc/cpuinfo | sort -u
vmx
$ lsmod | grep kvm
kvm_intel             380928  0
kvm                  1142784  1 kvm_intel
```

`vmx` means Intel VT-x is present and `svm` means AMD-V. If neither appears, the feature may be off in the firmware settings, or you are already inside a virtual machine that does not pass it through.

Type 1 and type 2

Kind	Runs on	Examples	Typical use
Type 1 (bare metal)	Directly on hardware	VMware ESXi, Microsoft Hyper-V, KVM, Xen, Proxmox VE	Servers and private clouds
Type 2 (hosted)	Inside a normal operating system	VirtualBox, VMware Workstation, Parallels	Laptops, testing

The line is not perfectly sharp. **KVM** is a part of the Linux kernel, so Linux itself becomes the hypervisor, which is why KVM is usually counted as type 1. **Proxmox VE** is a Debian-based system that bundles KVM virtual machines, containers, clustering and a web interface.

The KVM stack

- **KVM** is the kernel module that uses the CPU features.
- **QEMU** is the user-space program that emulates the rest of the machine: devices, disks, network cards, and it is the process you see for each guest.
- **libvirt** is the management layer. Tools such as `virsh` and `virt-manager` talk to libvirt, which starts QEMU with the right options.
- **virtio** is a family of paravirtual drivers. A guest with virtio disk and network drivers is much faster than one that uses emulated hardware.

MARGA by EknathaLabs marga.eknathalabs.com

```
$ virsh list --all
 Id   Name        State
-----------------------------
 3    web-01      running
 -    batch-02    shut off
```

Virtual machines and containers

A virtual machine includes a full guest kernel. A container shares the host kernel and isolates processes with namespaces and cgroups. That makes containers lighter and faster to start, and virtual machines a stronger boundary because an attacker has to escape a hypervisor, not just a shared kernel. Many platforms run containers inside virtual machines to get both.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How What virtualization is and the types of hypervisor fits together

Part	What it does	What goes wrong
What a hypervisor does	A hypervisor is the software that creates virtual machines and hands each one a share of CPU, memory, disk and network.	Forgetting that nested virtualization (a hypervisor inside a guest) needs the setting enabled on the outer host.
Type 1 and type 2	The line is not perfectly sharp. KVM is a part of the Linux kernel, so Linux itself becomes the hypervisor, which is why KVM is usually counted as type 1.	Judging a host by CPU use alone and ignoring memory, which is usually the first limit.
The KVM stack	KVM is the kernel module that uses the CPU features.	Running guests on emulated disk and network devices instead of virtio.
Virtual machines and containers	A virtual machine includes a full guest kernel.	Treating a virtual machine as a security boundary that needs no patching.

Flow: What a hypervisor does to Type 1 and type 2 / Type 1 and type 2 to The KVM stack / The KVM stack to Virtual machines and containers

Incident: The guest that would not start

Incident: *A new KVM host refused to start any guest and the team assumed the image was broken.*

```
$ virsh start web-01
error: Failed to start domain 'web-01'
error: unsupported configuration: Domain requires KVM, but it is not available. Check that virtual \
    ization is enabled in the host BIOS, and host configuration is setup to load the kvm modules.
$ lsmod | grep kvm
$ grep -c vmx /proc/cpuinfo
0
```

The image was fine. Hardware virtualization was switched off in the server firmware, so the CPU did not advertise `vmx` and the kvm module could not load. After the setting was enabled and the host restarted, the same command worked. When a guest will not start, check the host before the guest.

Common mistakes

- Judging a host by CPU use alone and ignoring memory, which is usually the first limit
- Running guests on emulated disk and network devices instead of virtio
- Forgetting that nested virtualization (a hypervisor inside a guest) needs the setting enabled on the outer host
- Treating a virtual machine as a security boundary that needs no patching

Check yourself

What do VT-x and AMD-V provide?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: CPU support that lets guest code run directly on the processor

Which part of the KVM stack emulates devices for a guest?

Answer: QEMU

Why is KVM usually called a type 1 hypervisor?

Answer: It is part of the Linux kernel, which runs directly on the hardware

What is the main isolation difference between a VM and a container?

Answer: A VM has its own kernel; a container shares the host kernel

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. CPU, memory and overcommit

15 minutes. Learn how vCPUs and memory are shared between guests, what overcommit means, and how to spot a crowded host.

Cloud and private cloud hosts rarely give every guest dedicated hardware. They promise more virtual CPUs and memory than exist, and rely on guests not asking for it all at once. That is **overcommit**. It saves money until the day it does not.

What you will learn

- How virtual CPUs are scheduled onto physical cores
- Memory overcommit, ballooning, KSM and swapping
- CPU ready time and steal time, the signs of a crowded host
- NUMA and why a large guest can run slower than a small one

Virtual CPUs

A **vCPU** is a thread of the host that runs a guest's CPU. If a host has 16 cores (32 with hyper-threading) and the guests together have 80 vCPUs, the ratio is 2.5 vCPUs per thread. Light workloads often tolerate 4 to 1 or more, busy production databases need close to 1 to 1.

When more vCPUs want to run than there are free threads, some wait. The guest sees this as **steal time**, shown as `st` in `top` inside a Linux guest. VMware shows the same idea as **CPU ready**.

```
$ top -bn1 | head -4 | tail -2
%Cpu(s): 22.0 us,  6.0 sy,  0.0 ni, 54.0 id,  0.0 wa,  0.0 hi,  0.0 si, 18.0 st
```

Eighteen percent steal means the guest wanted CPU and the host could not give it. The guest looks idle at 54 percent, yet it is slow. The fix is on the host: move guests away, reduce vCPU counts, or add hardware.

Memory

Memory is the harder resource, because a guest that is out of memory fails instead of slowing down.

- **Ballooning** lets the host ask a guest driver to take memory back. The guest swaps or drops cache to comply.
- **KSM (kernel samepage merging)** shares identical pages between guests. It saves memory and costs some CPU.
- **Host swap** is the last resort. A guest whose memory is swapped by the host becomes very slow, because the guest does not know.
- **Huge pages** reduce overhead for large guests such as databases.

```
$ free -m
               total        used        free      shared  buff/cache   available
Mem:          262144      249910        1210         312       11024        9800
Swap:          32768       14200       18568
```

When the host itself starts using swap with guests running, memory is overcommitted too far. A common rule is to keep memory overcommit near 1 to 1 for production, and to allow more only where guests are idle and can be paused.

NUMA

Large servers have several CPU sockets, each with memory attached. Reading memory from another socket is slower. A guest bigger than one socket may span both and lose speed. Where possible, pin large guests to one NUMA node, or size them to fit.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How CPU, memory and overcommit fits together

Part	What it does	What goes wrong
Virtual CPUs	A vCPU is a thread of the host that runs a guest's CPU.	Overcommitting memory on a host that runs databases.
Memory	Memory is the harder resource, because a guest that is out of memory fails instead of slowing down.	Giving every guest many vCPUs "to be safe": more vCPUs can make scheduling worse.
NUMA	Large servers have several CPU sockets, each with memory attached.	Starting a guest larger than a NUMA node without pinning.

Flow: Virtual CPUs to Memory / Memory to NUMA

Incident: The Monday morning slowdown

Incident: *Every Monday at 09:00 the web guests slowed, yet their graphs showed low CPU.*

```
guest$ top -bn1 | grep Cpu
%Cpu(s): 15.0 us,  4.0 sy,  0.0 ni, 40.0 id,  0.0 wa,  0.0 hi,  0.0 si, 41.0 st
host$ virsh nodeinfo | grep -E 'CPU\(s\)|Memory'
CPU(s):              32
Memory size:         268435456 KiB
host$ virsh list | wc -l
39
```

Forty-one percent steal showed the guests were waiting for the host. Thirty-six running guests shared 32 threads, and a weekly batch of reports started at 09:00 on the same host. The team moved the batch guests to a separate host and set a limit of 3 vCPUs per thread for that cluster. Guest graphs inside the guest can hide this problem. Steal time and host CPU tell the truth.

Common mistakes

- Giving every guest many vCPUs "to be safe": more vCPUs can make scheduling worse
- Overcommitting memory on a host that runs databases
- Reading only guest CPU graphs and ignoring steal or ready time
- Starting a guest larger than a NUMA node without pinning

Check yourself

What does 18 percent steal time inside a guest mean?

Answer: The guest wanted CPU but the host could not run it at that moment

What is memory ballooning?

Answer: The host asks a guest driver to give memory back

Why can a guest larger than one NUMA node be slower?

Answer: Some memory is attached to another socket and is slower to reach

Which workload should have close to 1 to 1 vCPU overcommit?

Answer: A busy production database

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. Virtual disks, snapshots and templates

14 minutes. Learn disk image formats, thin provisioning, how snapshot chains grow, and how templates and clones save time.

A virtual machine's disk is a file or a volume on the host. How that file is stored decides speed, space use and what happens when you take a snapshot.

What you will learn

- Common image formats: raw, qcow2, VMDK, VHDX
- Thin and thick provisioning
- How a snapshot works and why long chains cause trouble
- Templates, clones and cloud-init

Image formats

Format	Used by	Notes
raw	KVM, many clouds	Simple and fast, no features of its own
qcow2	KVM and QEMU	Thin provisioning, snapshots, compression
VMDK	VMware	Thin or thick, snapshots as delta files
VHDX	Hyper-V	Dynamic or fixed

```
$ qemu-img info web-01.qcow2
image: web-01.qcow2
file format: qcow2
virtual size: 100 GiB (107374182400 bytes)
disk size: 8.2 GiB
```

The **virtual size** is what the guest sees. The **disk size** is what the file takes on the host. That gap is **thin provisioning**: space is taken only as the guest writes. It saves space and lets you promise more than you have, which is another kind of overcommit. If every guest fills its disk, the storage pool runs out and guests fail with write errors.

Snapshots

A snapshot freezes the disk at a point in time. New writes go to a separate file, called a delta or overlay, and the original becomes read-only. Rolling back means discarding the delta.

```
$ virsh snapshot-create-as web-01 before-upgrade
Domain snapshot before-upgrade created
$ virsh snapshot-list web-01
 Name             Creation Time               State
------------------------------------------------------
 before-upgrade   2026-11-02 10:14:09 +0000   shutoff
```

A snapshot is not a backup. It lives on the same storage as the disk, so losing the storage loses both. Each snapshot adds a layer: reads may have to look through several files, which slows the guest, and every delta uses space. A **chain** of dozens of snapshots can fill a pool and make deleting them slow because deltas must be merged.

Use snapshots for short-lived safety, such as before an upgrade, and delete them when the change is accepted.

MARGA by EknathaLabs marga.eknathalabs.com

Templates and clones

A **template** is a prepared guest image with the operating system installed and cleaned. A **full clone** copies the disk. A **linked clone** uses the template as a read-only base and stores only the differences, so it is fast and small, but depends on the base.

cloud-init configures a guest at first boot from user data: host name, users, SSH keys, packages. It lets one generic template serve many roles.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Virtual disks, snapshots and templates fits together

Part	What it does	What goes wrong
Image formats	The virtual size is what the guest sees.	Overcommitting thin disks with no alert on pool free space.
Snapshots	A snapshot freezes the disk at a point in time.	Cloning a guest without changing its host name, machine id and SSH host keys.
Templates and clones	A template is a prepared guest image with the operating system installed and cleaned.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Image formats to Snapshots / Snapshots to Templates and clones

Incident: The pool that filled overnight

Incident: *A pool that was 60 percent full on Friday reached 100 percent on Sunday and every guest paused.*

```
$ virsh pool-info vmstore
Capacity:       500.00 GiB
Allocation:     499.98 GiB
Available:      0.02 GiB
$ virsh snapshot-list db-01 | wc -l
42
```

A backup tool created a snapshot of `db-01` every hour and never removed them. Forty snapshots each held the changes of an hour of database writes. Deleting the old snapshots merged the files and freed 180 GiB, and the guests resumed. The team added a retention rule and an alert at 80 percent pool use.

Common mistakes

- Treating snapshots as backups
- Leaving snapshots for weeks
- Overcommitting thin disks with no alert on pool free space
- Cloning a guest without changing its host name, machine id and SSH host keys

Check yourself

What is the difference between virtual size and disk size of a qcow2 image?

Answer: Virtual size is what the guest sees, disk size is what the file uses on the host

Why is a snapshot not a backup?

Answer: It sits on the same storage as the disk, so losing the storage loses both

What is a linked clone?

Answer: A clone that reads from a template base and stores only its changes

What is cloud-init used for?

Answer: Configuring a guest at first boot, such as users, keys and host name

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Virtual networking: bridges, VLANs and NAT

14 minutes. See how guests get network access through bridges, NAT and VLANs, and how to find why a guest has no connectivity.

A guest needs a network card, and the card needs somewhere to plug in. The hypervisor provides a virtual switch for that, and the choices you make there decide who can reach the guest and what it can reach.

What you will learn

- Virtual NICs and the virtual switch
- Bridged, NAT and isolated networks
- VLAN tagging on the host uplink
- SR-IOV and when to skip the virtual switch

The virtual switch

Each guest has one or more **virtual NICs**, usually virtio. On the host they appear as interfaces such as `vnet3` attached to a **bridge** or a vSwitch. On Linux this is often the `br0` bridge or an Open vSwitch bridge. VMware calls it a vSwitch, and Hyper-V a virtual switch.

```
$ ip -br link show master br0
eno1      UP    3c:ec:ef:11:22:33
vnet0     UP    fe:54:00:aa:bb:01
vnet1     UP    fe:54:00:aa:bb:02
```

The physical port `eno1` and the guest ports `vnet0` and `vnet1` are on one bridge, so the guests behave like machines plugged into the same switch as the host.

Network modes

- **Bridged**: guests have addresses on the physical network and are visible to it. Common for servers.
- **NAT**: the host hides guests behind its own address. Guests can go out, but nothing can reach them unless you forward ports. This is libvirt's default `virbr0` network, convenient for desktops and tests.
- **Isolated**: guests can reach each other and the host but not the outside.
- **Routed**: the host routes between a guest network and the physical one without translation.

VLANs

A **VLAN** splits one physical network into separate broadcast domains using a tag in the Ethernet frame. The switch port to the host is often a **trunk** that carries several VLANs, and the host or hypervisor assigns each guest to one. If the trunk does not allow the tag, or the guest is on the wrong tag, the guest has a link but no connectivity.

```
$ bridge vlan show
port    vlan-id
eno1    1 PVID Egress Untagged
        20
        30
vnet0   20 PVID Egress Untagged
vnet1   30 PVID Egress Untagged
```

SR-IOV

SR-IOV lets one physical NIC present several virtual functions that are handed directly to guests. It skips the software switch, so latency and CPU use drop. The cost is flexibility: live migration and firewall rules at the host become harder.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Virtual networking: bridges, VLANs and NAT fits together

Part	What it does	What goes wrong
The virtual switch	Each guest has one or more virtual NICs, usually virtio.	Two guests with the same MAC address after a clone.
Network modes	Bridged: guests have addresses on the physical network and are visible to it.	Putting a guest on a NAT network and then wondering why clients cannot reach it.
VLANs	A VLAN splits one physical network into separate broadcast domains using a tag in the Ethernet frame.	Forgetting that a trunk port must allow every VLAN used.
SR-IOV	SR-IOV lets one physical NIC present several virtual functions that are handed directly to guests.	Disabling the firewall on the host because a guest could not connect, without checking the VLAN first.

Flow: The virtual switch to Network modes / Network modes to VLANs / VLANs to SR-IOV

Incident: A new guest with a link and no network

Incident: *A guest came up, got no address from DHCP, and the team blamed the DHCP server.*

```
guest$ ip -br addr
ens3   UP   fe80::5054:ff:fe12:3456/64
host$ bridge vlan show dev vnet5
port    vlan-id
vnet5   40 PVID Egress Untagged
host$ bridge vlan show dev eno1
port    vlan-id
eno1    1 PVID Egress Untagged
        20
        30
```

The guest was assigned to VLAN 40, but the uplink `eno1` only carried VLANs 20 and 30. The DHCP server was fine. After the network team added VLAN 40 to the trunk, the guest received an address. Check each hop of the path: guest, vnet port, bridge, uplink, switch.

Common mistakes

- Putting a guest on a NAT network and then wondering why clients cannot reach it
- Forgetting that a trunk port must allow every VLAN used
- Two guests with the same MAC address after a clone
- Disabling the firewall on the host because a guest could not connect, without checking the VLAN first

Check yourself

What does a bridge do for guests?

Answer: It connects guest virtual NICs and the physical port like a switch

Why can a guest on libvirt's default NAT network not be reached from outside?

Answer: The host hides it behind its address unless ports are forwarded

A guest has a link but no DHCP address on VLAN 40. What is a likely cause?

Answer: The uplink trunk does not carry VLAN 40

What is the main trade-off of SR-IOV?

Answer: Lower latency and CPU use, but less flexibility such as live migration

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Live migration, high availability and troubleshooting

15 minutes. Understand how guests move between hosts without stopping, how HA restarts them after a failure, and how to diagnose common faults.

A host needs patching, or fails. The point of a virtual platform is that guests do not have to suffer for it.

What you will learn

- Live migration and what it needs
- High availability, fencing and split brain
- Capacity planning for failure (N+1)
- A checklist for the faults you will meet

Live migration

Live migration (VMware vMotion, KVM live migration) copies a running guest's memory from one host to another while it keeps running, then switches over in a short pause of tens of milliseconds. It makes host maintenance invisible to users.

Requirements usually include:

- A shared or replicated storage that both hosts can reach, or storage migration that copies the disk too
- Compatible CPU features: a guest that uses a new instruction cannot move to an older CPU, so clusters set a common **CPU model**
- A fast network between hosts for the memory copy
- Matching networks on both hosts, so the guest keeps its connectivity

```
$ virsh migrate --live --persistent web-01 qemu+ssh://host-b/system
$ virsh list
 Id   Name   State
-------------------
```

A guest that dirties memory faster than the network can copy it may never converge. Throttling the guest, or using post-copy migration, solves that.

High availability

HA restarts guests on another host after a host failure. It differs from migration: the guest does restart, with a short outage. The cluster must be sure the failed host is really off before it starts the same guest elsewhere, otherwise two copies write to one disk and corrupt it. That is the danger of **split brain**. **Fencing** (also called STONITH) cuts power or access to a suspect host to make sure it is down.

A cluster also needs **quorum**: a majority of nodes must agree on who is alive. With two nodes and no tie-breaker, a network split leaves each side unsure.

Plan for failure

Spare capacity must exist for the guests that restart. **N+1** means the cluster can lose one host and still run everything. If every host runs at 85 percent memory, a failed host leaves guests with nowhere to go.

Troubleshooting checklist

1. Is hardware virtualization on, and is `kvm` loaded?
2. Is there free memory on the host for the guest, with overcommit counted?
3. Is there free space in the storage pool, including snapshot deltas?
4. Is the disk file present, with the right permissions and SELinux or AppArmor labels?

MARGA by EknathaLabs
marga.eknathalabs.com

5 Is the guest on the right network and VLAN?

6 Are guests waiting for CPU (steal or ready time)?

```
$ journalctl -u libvirtd --since "10 min ago" | tail -3
libvirtd[1043]: internal error: qemu unexpectedly closed the monitor
libvirtd[1043]: cannot set up guest memory 'pc.ram': Cannot allocate memory
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Live migration, high availability and... fits together

Part	What it does	What goes wrong
Live migration	Live migration (VMware vMotion, KVM live migration) copies a running guest's memory from one host to another while it keeps running, then switches over in a short pause...	Running clusters at high memory use and calling it efficient.
High availability	HA restarts guests on another host after a host failure.	HA with no fencing.
Plan for failure	Spare capacity must exist for the guests that restart.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Troubleshooting checklist	Is hardware virtualization on, and is kvm loaded?	Using a snapshot as the way to move a guest.

Flow: Live migration to High availability / High availability to Plan for failure / Plan for failure to Troubleshooting checklist

Incident: Maintenance that moved the problem

Incident: *Draining a host for patching started a chain of guest failures on the next host.*

```
$ virsh -c qemu+ssh://host-b/system nodeinfo | grep Memory
Memory size:         134217728 KiB
$ virsh -c qemu+ssh://host-b/system list | wc -l
19
$ virsh migrate --live batch-07 qemu+ssh://host-b/system
error: internal error: process exited while connecting to monitor: Cannot allocate memory
```

Host B already used almost all of its memory. The migration failed, and then the HA service on host B restarted two guests during the same minute and exhausted memory. The team set a rule that a host may hold at most 80 percent of its memory with guests, so one host can always be drained into the rest, and drained one host at a time.

Common mistakes

- Running clusters at high memory use and calling it efficient
- Mixing CPU generations without a common CPU model
- HA with no fencing
- Using a snapshot as the way to move a guest

Check yourself

What must live migration copy while the guest keeps running?

Answer: Its memory, changing pages included

Why does a cluster need fencing?

Answer: To be sure a failed host is really off before restarting its guests elsewhere

What does N+1 capacity mean?

Answer: The cluster can lose one host and still run every guest

MARGA by EknathaLabs marga.eknathalabs.com

A guest dirties memory faster than the network copies it. What happens in live migration?

Answer: It may never finish unless the guest is throttled or post-copy is used

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Virtualization operations: capacity, overcommit and noisy neighbours

12 minutes. Plan host capacity with sensible overcommit ratios, spot CPU steal, memory ballooning and I/O contention, and place virtual machines so one guest cannot starve others.

A hypervisor host shares CPU, memory, disk and network among guests. Overcommit is what makes virtualization economical, and contention is its price. Operations means choosing ratios with evidence, watching the right signals and moving guests before users feel pain.

What you will learn

- CPU, memory and storage overcommit and safe starting ratios
- Signals of contention: steal, ready time, ballooning, swapping and I/O wait
- NUMA, pinning and huge pages for demanding guests
- Placement and limits that contain noisy neighbours

Overcommit by resource

- **CPU** can be overcommitted well, because most guests are idle most of the time. Common starting points are 4:1 virtual to physical cores for general workloads, lower (1:1 or 2:1) for latency-sensitive or heavy compute guests. Measure the real contention, not the ratio.
- **Memory** is risky to overcommit. When the host runs out, it swaps or kills guests. Use modest ratios (often 1.0 to 1.5), enable ballooning and page sharing only with monitoring, and keep a reserve for the host.
- **Storage** can be thin-provisioned, but the pool must be watched, since a full pool freezes many guests at once.

Treat these as starting points to tune with data, because the right values depend on your workloads.

Detect contention

Inside the guest: high **steal time** (`%st` in `top` or `vmstat`) means the guest wanted CPU and the host did not give it. On the host:

```
$ virsh nodecpustats
$ virsh domstats --cpu-total --balloon --block --vcpu vm-web-01
$ virsh domblkstat vm-web-01 vda
$ virsh domifstat vm-web-01 vnet3
$ numastat -m | head -20
$ cat /proc/pressure/cpu
```

The pressure stall information in `/proc/pressure/` shows how much time tasks wait for CPU, memory or I/O on the host. A host whose CPU pressure rises while guest steal rises is oversubscribed. Memory pressure plus swap use on the host is an emergency, because every guest suffers.

NUMA, pinning and huge pages

Large hosts have several NUMA nodes, each with local memory. A guest whose vCPUs run on one node but whose memory sits on another pays a latency cost. For performance-sensitive guests, keep vCPUs and memory on the same node (NUMA-aware placement), pin vCPUs to dedicated cores, and use huge pages to reduce page table overhead. These guests give up flexibility for predictability, so reserve them a dedicated host pool and do not overcommit it.

MARGA by EknathaLabs marga.eknathalabs.com

Contain noisy neighbours

Set limits and shares so a single guest cannot take everything: CPU shares and quotas, disk I/O throttling (IOPS and bandwidth) and network rate limits through libvirt or the cloud layer's flavors. Separate workload classes by host aggregate: a general pool with overcommit, a premium pool without it, a storage-heavy pool. Keep headroom to evacuate the largest host (N+1) and move guests early using live migration when a host trends toward contention.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Virtualization operations: capacity, overcommit... fits together

Part	What it does	What goes wrong
Overcommit by resource	CPU can be overcommitted well, because most guests are idle most of the time.	Mixing latency-sensitive and batch guests on one pool.
Detect contention	Inside the guest: high steal time (%st in top or vmstat) means the guest wanted CPU and the host did not give it.	Overcommitting memory like CPU.
NUMA, pinning and huge pages	Large hosts have several NUMA nodes, each with local memory.	No spare capacity to evacuate a failed host.
Contain noisy neighbours	Set limits and shares so a single guest cannot take everything: CPU shares and quotas, disk I/O throttling (IOPS and bandwidth) and network rate limits through libvirt...	Judging by the ratio instead of measured contention.

Flow: Overcommit by resource to Detect contention / Detect contention to NUMA, pinning and huge pages / NUMA, pinning and huge pages to Contain noisy neighbours

Incident: The Monday morning stalls

Incident: *Every Monday at 09:00 several unrelated application servers on the same host became slow for about 20 minutes.*

```
$ virsh domstats --cpu-total vm-batch-07 | grep cpu.time
cpu.time=48230193847211
$ cat /proc/pressure/cpu
some avg10=62.40 avg60=58.12 avg300=40.77 total=19238421831
```

A batch guest, scheduled to run weekly reports at 09:00, had 16 vCPUs and used all of them, while the host had 40 cores for 160 vCPUs of other guests. The other guests saw steal times above 20 percent. The team moved the batch guest to a dedicated aggregate with its own cores, set CPU shares, and added an alert on host CPU pressure and guest steal.

Common mistakes

- Overcommitting memory like CPU
- Judging by the ratio instead of measured contention
- Mixing latency-sensitive and batch guests on one pool
- No spare capacity to evacuate a failed host

Check yourself

What does high steal time inside a guest indicate?

Answer: The host is not providing the CPU the guest requests

Which resource is riskiest to overcommit?

Answer: Memory

Why use NUMA-aware placement?

Answer: Keeps vCPUs near their memory to avoid remote-access latency

How do you stop a noisy guest from starving others?

Answer: Apply CPU, disk I/O and network limits and separate pools by workload

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Virtualization operations: libvirt and KVM day-two work

13 minutes. Run KVM hosts with libvirt day to day: inspect and change guests, take snapshots and backups safely, patch hosts with live migration, and keep configuration consistent.

After the first guests are running, the work is routine and risky in equal parts: adding disks, changing memory, taking snapshots before changes, patching the host without downtime and keeping many hosts consistent. These tasks have safe, repeatable procedures.

What you will learn

- Inspect and adjust guests with `virsh`
- Snapshots and backups, and their limits
- Patching hosts with live migration
- Keeping host configuration consistent

Know the state

```
$ virsh list --all
$ virsh dominfo vm-web-01
$ virsh domblklist vm-web-01
$ virsh dumpxml vm-web-01 | head -40
$ virsh pool-list --details
$ virsh net-list --all
```

The domain XML is the source of truth for a guest: CPUs, memory, disks, NICs, devices. Treat it as configuration: keep copies, change it with `virsh edit` or an automation tool, and know that some changes apply only after a guest restart (use `--config` for persistent changes and `--live` for the running guest).

```
$ virsh setmem vm-web-01 4G --config --live
$ virsh attach-disk vm-web-01 /var/lib/libvirt/images/data2.qcow2 vdb --persistent --subdriver qco \
    w2
```

Snapshots and backups

A snapshot captures a guest's disk state (and optionally memory) at a moment. Internal snapshots live in the qcow2 file, external ones create an overlay file. Snapshots are good for short-term protection before a change, but long chains slow I/O and are fragile, and a snapshot on the same storage is not a backup.

```
$ virsh snapshot-create-as vm-web-01 pre-upgrade --disk-only --atomic --quiesce
$ virsh snapshot-list vm-web-01
$ virsh blockcommit vm-web-01 vda --active --pivot
```

`--quiesce` asks the guest agent to freeze the filesystem for a consistent image, which needs `qemu-guest-agent` in the guest. After the change is confirmed, merge the overlay back (blockcommit) and delete old snapshot metadata. For real backups, copy consistent images or use backup APIs to a separate system, and test restoring a guest on another host.

Patching a host without downtime

1. Check capacity: the other hosts must have room for the guests on this one.
2. Mark the host unavailable for new guests (in a cloud layer, disable the compute service).
3. **Live migrate** guests away.
4. Patch and reboot, check, and put the host back.

```
$ virsh migrate --live --persistent --undefinesource vm-web-01 qemu+ssh://kvm02/system
$ virsh domjobinfo vm-web-01
```

Live migration needs compatible CPU models, shared or migratable storage, a stable network between hosts and enough bandwidth to copy dirty memory faster than the guest changes it. Guests with very high

MARGA by EknathaLabs marga.eknathalabs.com

write rates may not converge; use auto-converge or post-copy options, or migrate in a quiet period.

Consistent hosts

Manage hosts as code: the same kernel and QEMU versions per pool, the same network and storage definitions, one source for libvirt networks and pools. Drift between hosts is the root of failed migrations ("works on kvm01, fails on kvm02").

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Virtualization operations: libvirt and KVM... fits together

Part	What it does	What goes wrong
Know the state	The domain XML is the source of truth for a guest: CPUs, memory, disks, NICs, devices.	Changing the running guest without --config.
Snapshots and backups	A snapshot captures a guest's disk state (and optionally memory) at a moment.	Treating a snapshot on the same storage as a backup.
Patching a host without downtime	Check capacity: the other hosts must have room for the guests on this one.	Live-migrating without checking CPU compatibility and capacity.
Consistent hosts	Manage hosts as code: the same kernel and QEMU versions per pool, the same network and storage definitions, one source for libvirt networks and pools.	Drift between hosts is the root of failed migrations ("works on kvm01, fails on kvm02").

Flow: Know the state to Snapshots and backups / Snapshots and backups to Patching a host without downtime / Patching a host without downtime to Consistent hosts

Incident: The snapshot chain

Incident: *A database guest became slower every week, and finally froze when its datastore filled.*

```
$ virsh snapshot-list db-prod-02 --tree | head
pre-upgrade-1
  |
  +- pre-upgrade-2
       |
       +- pre-patch
            |
            +- pre-schema-change
$ qemu-img info --backing-chain /var/lib/libvirt/images/db-prod-02.pre-schema-change | grep -c "im \
    age:"
9
```

Each change had added a snapshot "just in case" and nobody merged them. Reads had to walk nine layers, writes created large overlays, and the datastore ran out of space. The team merged the chain in a maintenance window, wrote a rule that snapshots live for 72 hours at most with an owner and a ticket, and added an alert on chain depth and datastore free space.

Common mistakes

- Treating a snapshot on the same storage as a backup
- Leaving snapshots for months
- Changing the running guest without `--config`
- Live-migrating without checking CPU compatibility and capacity

Check yourself

What does --config do in virsh commands?

Answer: Makes the change persistent for the next boot

MARGA by EknathaLabs
marga.eknathalabs.com

Why install the guest agent for snapshots?

Answer: It lets libvirt freeze the filesystem for a consistent image

What problem do long snapshot chains cause?

Answer: Slower I/O, fragile merges and wasted space

Before patching a host, what must you check?

Answer: That other hosts have capacity for its guests

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Virtualization operations: guest performance troubleshooting

12 minutes. Find whether slowness comes from the guest, the host, storage or the network, using paired views from inside the guest and on the hypervisor.

"The VM is slow" can mean the application, the guest operating system, the hypervisor, shared storage or the network. The fastest way to find the layer is to compare two views of the same moment: what the guest sees and what the host sees.

What you will learn

- A layered method for guest slowness
- Paired guest and host checks for CPU, memory, disk and network
- Virtio drivers, guest agents and common misconfigurations
- When to move, resize or fix the guest

Method: top-down, then bottom-up

1. **Define the symptom** and the time window. Does it affect one guest, many guests on one host, or many hosts?
2. **Inside the guest**: check the USE signals (CPU, memory, disk, network), the application and the kernel log.
3. **On the host**: check the same resources for this guest and for the host overall.
4. **Shared layers**: storage latency, network paths and the management plane.

If many guests on one host slow down together, suspect the host or its storage. If only one guest slows down, look at its own configuration or load first.

CPU and memory pairs

Guest signal	Host check
High `%steal`	Host CPU pressure, vCPU to core ratio
High load, low CPU use	Guest I/O wait; host disk latency
Memory pressure, swapping	Balloon size, host free memory
Single vCPU saturated	Guest app is single-threaded

```
$ virsh domstats --balloon --cpu-total vm-api-03
$ virsh vcpuinfo vm-api-03
$ virt-top -n 1
```

If the balloon driver has taken memory from the guest (`balloon.current` below `balloon.maximum`) while the guest swaps, the host is short on memory.

Disk and network pairs

```
$ iostat -xz 1 3
$ virsh domblkstat vm-api-03 vda --human
$ virsh domiflist vm-api-03
$ virsh domifstat vm-api-03 vnet5
```

Compare the guest's `await` with the latency at the storage layer. A large gap points to the hypervisor's I/O path (queueing, throttles); both high points at storage. For network, a growing `drop` count on the host's `vnet` interface while the guest sends normally means the host side is dropping, perhaps because of a saturated bridge, a full qdisc or a throttle. Use **virtio** drivers (`virtio-blk` or `virtio-scsi`, `virtio-net`) with multiqueue for busy guests; emulated devices such as IDE or e1000 are much slower, and are a classic silent cause.

MARGA by EknathaLabs marga.eknathalabs.com

Configuration checks

Check the XML for a sensible CPU model (host-passthrough or a modern model for performance), `cache` and `io` modes on disks, the machine type, whether the guest agent is connected, and clock settings. Time drift in guests can break certificates and clustering, so use `chrony` with the KVM clock.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Virtualization operations: guest performance... fits together

Part	What it does	What goes wrong
Method: top-down, then bottom-up	Define the symptom and the time window. Does it affect one guest, many guests on one host, or many hosts?	Diagnosing only inside the guest.
CPU and memory pairs	If the balloon driver has taken memory from the guest (balloon.current below balloon.maximum) while the guest swaps, the host is short on memory.	Ignoring balloon and host memory pressure.
Disk and network pairs	Compare the guest's await with the latency at the storage layer.	Leaving emulated devices on busy guests.
Configuration checks	Check the XML for a sensible CPU model (host-passthrough or a modern model for performance), cache and io modes on disks, the machine type, whether the guest agent is...	Resizing a guest before locating the bottleneck.

Flow: Method: top-down, then bottom-up to CPU and memory pairs / CPU and memory pairs to Disk and network pairs / Disk and network pairs to Configuration checks

Incident: The slow network inside the hypervisor

> **Incident:** *File transfers between two guests on the same host topped out at a small fraction of what the 10 Gb/s network should allow.*

```
$ virsh dumpxml vm-files-01 | grep -A3 "<interface"
<interface type='bridge'>
  <model type='e1000'/>
$ ethtool -S vnet4 | grep -i drop
     rx_dropped: 183921
```

The guest used the emulated `e1000` network card, which consumes a lot of CPU in the hypervisor, and a single-queue configuration. After changing the model to `virtio` with multiqueue and installing the paravirtual driver in the guest, throughput grew several times and the drops disappeared. The team made virtio the default in their guest templates and added a check for emulated devices to their audit scripts.

Common mistakes

- Diagnosing only inside the guest
- Leaving emulated devices on busy guests
- Ignoring balloon and host memory pressure
- Resizing a guest before locating the bottleneck

Check yourself

What is the best way to locate a performance problem in a VM?

Answer: Compare what the guest sees with what the host sees

Which network model is generally faster for guests?

Answer: virtio-net

MARGA by EknathaLabs marga.eknathalabs.com

Many guests on one host are slow together. What does that suggest?

Answer: A host-level or shared-resource problem

What does a shrunk balloon with guest swapping suggest?

Answer: The host is short on memory and reclaimed it from the guest

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. VM backup, replication and recovery

12 minutes. Back up virtual machines with snapshots and agents, replicate to another site and test restores against recovery objectives.

Virtual machines make copying easy and recovery tempting to take for granted. The details of consistency, storage location and restore speed decide whether a backup actually saves you.

What you will learn

- Snapshots versus backups
- Application-consistent backups and guest agents
- Replication and failover to a second site
- Restore testing and retention

Snapshots and backups

A **snapshot** records a VM's disk state at a moment and stores changes separately. It lives on the same storage and slows down as it grows, so use it for short-lived rollback points before a change. A **backup** is a copy on separate storage with retention, and it survives loss of the primary storage.

```
virsh snapshot-create-as web-01 pre-upgrade --disk-only --atomic
virsh snapshot-list web-01
virsh blockcommit web-01 vda --active --pivot     # merge the snapshot back when done
```

Consistency

A crash-consistent copy is like pulling the power: filesystems usually recover, but databases may need repair. For **application-consistent** backups, use the guest agent to freeze the filesystem or tell the application to flush data.

```
virsh domfsfreeze web-01
# take the storage snapshot or copy
virsh domfsthaw web-01
```

Keep the freeze short, and use database-native backups for large critical databases.

Replication

Replicate changed blocks to another site on a schedule or continuously, so that a failover boots a recent copy. Know your **RPO**, set by replication lag, and **RTO**, set by how fast the copy can boot and the network can be reconfigured.

Restore tests

Schedule restores into an isolated network, boot the VM, check that the service answers and record the time. Keep retention tiers such as daily, weekly and monthly, and protect some copies from deletion, for example with immutable storage.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How VM backup, replication and recovery fits together

Part	What it does	What goes wrong
Snapshots and backups	A snapshot records a VM's disk state at a moment and stores changes separately.	Backups on the same storage as the VMs.
Consistency	A crash-consistent copy is like pulling the power: filesystems usually recover, but databases may need repair.	Ignoring application consistency.
Replication	Replicate changed blocks to another site on a schedule or continuously, so that a failover boots a recent copy.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Restore tests	Schedule restores into an isolated network, boot the VM, check that the service answers and record the time.	Never testing a restore.

Flow: Snapshots and backups to Consistency / Consistency to Replication / Replication to Restore tests

Incident: The snapshot that filled the datastore

Incident: *A long-lived snapshot grew until it filled storage.*

```
datastore: 100% used
web-01 snapshot delta: 1.2 TB, snapshot age 71 days
VMs paused: 38
```

Someone had used a snapshot as a backup before an upgrade and forgot to remove it. The delta grew until the datastore filled and VMs paused. The team limited snapshot age by policy, added alerts and moved to scheduled backups on separate storage.

Common mistakes

- Using snapshots as backups
- Backups on the same storage as the VMs
- Never testing a restore
- Ignoring application consistency

Check yourself

What is the difference between snapshot and backup?

Answer: A backup lives on separate storage

What does a guest agent help with?

Answer: Application-consistent copies

What determines RPO in replication?

Answer: Replication lag

How know a backup works?

Answer: Restore it and test the service

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. VM templates, cloud-init and image pipelines

13 minutes. Build clean, patched VM images in a pipeline, use cloud-init for per-instance settings and avoid hand-built templates that drift.

Every VM starts from an image. If the image is a mystery, every VM is a mystery. Treat images like software: built from source, tested and versioned.

What you will learn

- Why hand-built templates drift
- Building images with Packer and configuration management
- Cleaning, hardening and scanning images
- Publishing, versioning and retiring images

A pipeline

```
source "qemu" "ubuntu" {
  iso_url          = "https://releases.ubuntu.com/24.04/ubuntu-24.04-live-server-amd64.iso"
  iso_checksum     = "sha256:REPLACE_WITH_CHECKSUM"
  output_directory = "output"
  disk_size        = "20G"
  ssh_username     = "builder"
  ssh_private_key_file = "~/.ssh/builder"
}

build {
  sources = ["source.qemu.ubuntu"]
  provisioner "ansible" {
    playbook_file = "playbooks/base-image.yml"
  }
  provisioner "shell" {
    script = "scripts/cleanup.sh"
  }
}

packer init . && packer validate . && packer build .
```

Pin the ISO checksum and run the build in CI with a clean builder. Use the same configuration management tool you use on servers, so images and running hosts follow one standard.

Clean and harden

```
cloud-init clean --logs --seed
truncate -s 0 /etc/machine-id
rm -f /etc/ssh/ssh_host_* /root/.bash_history
apt-get clean && fstrim -av
```

Remove machine IDs, SSH host keys, logs and history so clones do not share identity. Apply a security baseline, disable unused services and scan the image for vulnerabilities and misconfiguration. Do not include secrets or personal SSH keys.

Identity at boot

Use **cloud-init** to set hostname, users, keys, network and join configuration on first boot, so one image serves every role.

Publish and retire

Test each image by booting it, checking that cloud-init completes and running a few smoke tests. Name images with a date or version, record the build commit and package list and keep only a few recent versions. Set a maximum age and rebuild monthly or when critical patches land.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How VM templates, cloud-init and image pipelines fits together

Part	What it does	What goes wrong
A pipeline	Pin the ISO checksum and run the build in CI with a clean builder.	Leaving machine-specific data in images.
Clean and harden	Remove machine IDs, SSH host keys, logs and history so clones do not share identity.	Clean and harden Remove machine IDs, SSH host keys, logs and history so clones do not share identity.
Identity at boot	Use cloud-init to set hostname, users, keys, network and join configuration on first boot, so one image serves every role.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Publish and retire	Test each image by booting it, checking that cloud-init completes and running a few smoke tests.	No smoke test for new images.

Flow: A pipeline to Clean and harden / Clean and harden to Identity at boot / Identity at boot to Publish and retire

Incident: Two hundred clones with the same host keys

Incident: *Cloned VMs shared SSH host keys because the template was not cleaned.*

```
web-01 host key SHA256:q1xA...
web-02 host key SHA256:q1xA...
identical host key on 212 hosts
```

The template had been converted without removing SSH host keys, so every clone kept them. Clients could not tell the hosts apart, and one compromised key mattered everywhere. The pipeline now cleans identity data, and a test checks that two instances never share a host key.

Common mistakes

- Templates edited by hand
- Leaving machine-specific data in images
- Never rebuilding for patches
- No smoke test for new images

Check yourself

What should a pipeline build images from?

Answer: Code and pinned sources

What must be removed before cloning?

Answer: SSH host keys and machine ID

What sets per-instance settings at boot?

Answer: cloud-init

How keep images fresh?

Answer: Rebuild on a schedule

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. GPU passthrough and device assignment

12 minutes. Give a VM direct access to a GPU or network card with VFIO and IOMMU, and understand groups, limitations and security.

Some workloads need real hardware speed, such as GPU compute or high-rate networking. Passthrough gives a VM direct control of a device, at the cost of the usual virtualization conveniences.

What you will learn

- IOMMU, VFIO and IOMMU groups
- Assigning a device to a VM
- Limits: migration, reset and sharing
- Security considerations

How it works

The **IOMMU** translates and isolates device memory access. **VFIO** is the kernel framework that lets user space, such as QEMU, own a device safely. Devices that share an **IOMMU group** must be assigned together, because the hardware cannot isolate them from each other.

```
# enable in kernel command line: intel_iommu=on iommu=pt  (or amd_iommu=on)
dmesg | grep -i -E "iommu|dmar"
for g in /sys/kernel/iommu_groups/*; do echo "group ${g##*/}:"; ls "$g/devices"; done
lspci -nnk -s 3b:00.0
```

Assign a device

```
echo "options vfio-pci ids=10de:2330" > /etc/modprobe.d/vfio.conf
update-initramfs -u && reboot
virsh nodedev-list --cap pci | grep 3b
```

```
<hostdev mode='subsystem' type='pci' managed='yes'>
  <source>
    <address domain='0x0000' bus='0x3b' slot='0x00' function='0x0'/>
  </source>
</hostdev>
```

With `managed='yes'`, libvirt binds the device to VFIO when the VM starts and returns it afterwards.

Limits

A passed-through device belongs to one VM at a time. **Live migration** is generally not possible with it attached. Reset behavior varies by device, so test repeated start and stop cycles. Large device memory needs matching guest settings, such as huge pages and a large BAR configuration.

Security

Assignment relies on the IOMMU for isolation. Keep firmware and drivers updated, restrict who can attach devices and prefer hardware partitioning, such as MIG or SR-IOV, when several tenants share a device.

```
virsh dumpxml gpu-vm | grep -A3 hostdev
nvidia-smi -L        # run inside the guest
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How GPU passthrough and device assignment fits together

Part	What it does	What goes wrong
How it works	The IOMMU translates and isolates device memory access.	Not checking IOMMU groups.
Assign a device	With managed='yes', libvirt binds the device to VFIO when the VM starts and returns it afterwards.	Expecting live migration with passthrough devices.
Limits	A passed-through device belongs to one VM at a time.	Skipping reset tests.
Security	Assignment relies on the IOMMU for isolation.	Sharing a passthrough device between tenants.

Flow: How it works to Assign a device / Assign a device to Limits / Limits to Security

Incident: The host that froze on shutdown

Incident: *A VM shutdown hung the whole host because the GPU could not reset.*

```
vfio-pci 0000:3b:00.0: not ready 65535ms after FLR; giving up
qemu-kvm: terminating on signal 15
host unresponsive for 4 minutes
```

The device did not support a clean function-level reset in that firmware version. The team updated the firmware, chose a different reset method, added a stop-and-start test to acceptance and wrote a runbook for recovering a stuck device.

Common mistakes

- Not checking IOMMU groups
- Expecting live migration with passthrough devices
- Skipping reset tests
- Sharing a passthrough device between tenants

Check yourself

What does the IOMMU provide?

Answer: Device memory isolation and translation

What is VFIO?

Answer: Framework for safe device access from user space

What must be assigned together?

Answer: Devices in the same IOMMU group

What is usually impossible with passthrough?

Answer: Live migration

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Hypervisor security and isolation

12 minutes. Harden hypervisors and guests: patch quickly, limit management access, isolate networks and storage, and plan for guest escape.

The hypervisor is the most privileged software in the stack. If a guest can reach or break it, the boundary between tenants disappears, so hardening it is part of daily operations.

What you will learn

- Reducing the attack surface of the host
- Isolating management, storage and guest networks
- Confinement and device emulation risks
- Patching, mitigations and trust separation

Minimal host

Install only what the hypervisor needs. Disable unused services and emulated devices in guests, such as floppy or legacy USB controllers, since device emulation code has historically contained escape bugs. Keep the host operating system, QEMU and firmware patched on a defined schedule.

Network separation

```
management network   : hypervisor admin, BMC       (jump host only)
storage network      : NFS, iSCSI, Ceph traffic     (hypervisors and storage only)
migration network    : live migration traffic       (hypervisors only)
guest networks       : VLANs per tenant or tier     (no route to the above)
```

Guests must never reach management or storage networks. Enforce this with VLANs, firewall rules and no routes. Protect live migration traffic with encryption if it crosses shared links.

Confinement

```
ps -eZ | grep qemu                  # SELinux labels per VM (sVirt)
virsh dumpxml web-01 | grep -A2 seclabel
```

sVirt or AppArmor profiles give each VM process its own label, so a compromised QEMU process cannot read other VMs' disks. Run QEMU as an unprivileged user.

Access and data

Restrict who can run `virsh` or use the management console, require strong authentication and log every action. Treat disk images and snapshots as sensitive data, because anyone who can copy an image has the machine's data. Encrypt storage where required.

Mitigations and trust

Follow vendor guidance on CPU side-channel mitigations and understand their performance cost. Do not run high-risk, untrusted workloads on the same hosts as sensitive ones. Use separate clusters or dedicated hosts for different trust levels.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Hypervisor security and isolation fits together

Part	What it does	What goes wrong
Minimal host	Install only what the hypervisor needs. Disable unused services and emulated devices in guests, such as floppy or legacy USB controllers, since device emulation code has...	Management interfaces reachable from guests.
Network separation	Guests must never reach management or storage networks.	Unpatched hypervisors.
Confinement	sVirt or AppArmor profiles give each VM process its own label, so a compromised QEMU process cannot read other VMs' disks.	Running QEMU as root.
Access and data	Restrict who can run virsh or use the management console, require strong authentication and log every action.	Mixing untrusted and sensitive workloads on one host.
Mitigations and trust	Follow vendor guidance on CPU side-channel mitigations and understand their performance cost.	Do not run high-risk, untrusted workloads on the same hosts as sensitive ones.

Flow: Minimal host to Network separation / Network separation to Confinement / Confinement to Access and data / Access and data to Mitigations and trust

Incident: The management port in the guest VLAN

Incident: *A guest could reach the hypervisor management interface.*

```
$ nmap -p 16509,22 10.60.0.1
22/tcp    open  ssh
16509/tcp open  libvirt
route to management network present
```

A flat network let guests see the hypervisor's management addresses. Nothing had been exploited, but the exposure was real. The team put management on a separate VLAN, removed routes from guest networks and added a recurring scan from a test guest that must fail.

Common mistakes

- Management interfaces reachable from guests
- Running QEMU as root
- Unpatched hypervisors
- Mixing untrusted and sensitive workloads on one host

Check yourself

What does sVirt do?

Answer: Confines each VM process with its own label

What must guests never reach?

Answer: Management and storage networks

Why treat images as sensitive?

Answer: They contain the machine's data

How handle mixed trust workloads?

Answer: Separate hosts or clusters

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Virtualization

virsh, qemu-img, bridges and migration commands, with the checks that explain a guest that will not start or a host that is slow.

Guests

```
virsh list --all
virsh start web-1
virsh shutdown web-1        # clean, asks the guest OS
virsh destroy web-1         # power off, like pulling the plug
virsh dominfo web-1
virsh console web-1
virsh domstate web-1 --reason
```

Warning: `destroy` *does not delete the guest, but it is an abrupt power-off. Use* `shutdown` *first.*

Memory and CPU

```
virsh nodememstats
virsh setmem web-1 4G --live
virsh setvcpus web-1 4 --live
virsh vcpuinfo web-1
free -m
grep -E "steal|st" <(top -bn1 | head -4)
```

Steal time above a few percent means the host CPU is oversubscribed.

Disks and snapshots

```
qemu-img info disk.qcow2
qemu-img create -f qcow2 -b base.qcow2 -F qcow2 overlay.qcow2
virsh pool-list
virsh pool-info default
virsh snapshot-list web-1
virsh snapshot-create-as web-1 before-upgrade
virsh snapshot-delete web-1 before-upgrade
```

Warning: *Never delete files in a pool by hand. Snapshot chains depend on their backing files.*

Network

```
virsh domiflist web-1
virsh net-list --all
ip link show type bridge
bridge link
bridge vlan show
```

Migration

```
virsh migrate --live web-1 qemu+ssh://host2/system
virsh domjobinfo web-1
virsh domjobabort web-1
```

If dirty rate is above bandwidth, the migration will not converge.

Quick diagnosis

Symptom	First look
Guest will not start	error text, free host memory, pool space
Guest paused (ioerror)	pool full or storage fault
Everything slow	host swap, steal time, disk latency
No network	`domiflist`, bridge exists, VLAN allowed on the switch

MARGA by EknathaLabs
marga.eknathalabs.com

Quick host checks

```
virsh list --all
virsh nodememstats
virsh domstats --cpu-total web-1
virsh pool-list --details
virsh domblklist web-1
```

Cautions

Danger: `virsh destroy` *powers a guest off instantly, like pulling the plug. Use* `virsh shutdown` *first.*

Warning: *Snapshots are not backups. They grow with every write and a full pool pauses all guests.*

Caution: *Overcommitting memory works until every guest is busy at once. Keep a reserve on the host.*

MARGA by EknathaLabs
marga.eknathalabs.com